

Universidad Politécnica de Madrid

Máster Universitario en Inteligencia Artificial



POLITÉCNICA

Artificial neural networks and deep learning Neural network model

Mario Gutiérrez López

Universidad Politécnica de Madrid

`mario.gutierrez.lopez@alumnos.upm.es`

Jesús Samuel Garcia Carballo

Universidad Politécnica de Madrid

`jesussamuel.garciacarballo@alumnos.upm.es`

Álvaro Merino Román

Universidad Politécnica de Madrid

`alvaro.merino.roman@alumnos.upm.es`

Madrid, 5 de enero de 2026

Índice de contenidos

1. Introducción	1
2. BBO: Biogeography-based Optimization Algorithm	2
2.1. Introducción	2
2.1.1. Migración	2
2.1.2. Mutación	3
2.2. Definiciones previas	4
2.3. Algoritmo	5
2.4. Diferencias entre BBO y otros algoritmos de optimización basados en poblaciones	6
2.5. Implementación	6
2.6. Conclusiones	7
3. Sistema de hormigas Elitista	10
3.1. Introducción	10
3.2. Sistema de Hormigas	10
3.3. Algoritmo	11
3.3.1. Estrategia elitista	13
3.4. Implementación	13
3.4.1. Descripción del Problema y Entorno	13
3.4.2. Enfoque mono-objetivo: (Elitist Ant System)	14
3.4.3. Extensión multi-objetivo (MOACO y MAS)	14
3.4.4. Resultados Experimentales	14
4. FANS: Fuzzy Adaptive Neighborhood Search	18
4.1. Introducción	18
4.1.1. Operador de Modificación	18
4.1.2. Valoración Difusa	18
4.1.3. Utilidad	19
4.1.4. Administrador de Operación	20
4.1.5. Administrador de Vecindario	20
4.1.6. Parámetros globales	20
4.2. Algoritmo	20
4.3. FANS como Heurística de Propósito General	21
5. Conclusiones	23

Índice de figuras

1.	Frentes de Pareto aproximados obtenidos por BBO-SOEA para las seis funciones de prueba T1 a T6.	9
2.	Comparativa entre la selección Greedy y la solución óptima. Sumando los mínimos absolutos viola la capacidad del O_1	15
3.	Frente de Pareto obtenido en el escenario de conflicto. Se observa la curva de compromiso entre el Coste Total y el Desequilibrio de Carga.	16
4.	Ejemplo de variaciones difusas.	22

1. Introducción

2. BBO: Biogeography-based Optimization Algorithm

2.1. Introducción

La Optimización Basada en Biogeografía (BBO) es un algoritmo de optimización inspirada en la biogeografía, que es el estudio de cómo se distribuyen geográficamente las especies biológicas.

Fue propuesto por Dan Simon en 2008 [1], como un método para resolver problemas de optimización complejos. BBO se basa en la idea de que las soluciones a un problema pueden ser representadas como "hábitats" en un ecosistema, y que la calidad de estas soluciones puede ser evaluada mediante un índice de idoneidad (HSI, por sus siglas en inglés).

Este algoritmo mantiene una población fija de soluciones y las modifica en cada iteración mediante dos operadores principales: migración y mutación.

2.1.1. Migración

Supongamos que tenemos un problema y una población de soluciones candidatas representada como un vector de números enteros. Cada uno de estos números enteros es considerado un SIV (*Suitability Index Variable*). Las soluciones que son buenas son consideradas habitantes con un HSI alto y las que son malas tienen un HSI bajo. Las soluciones con un HSI alto representan hábitats de muchas especies mientras las soluciones con un HSI bajo representan hábitats de pocas especies. Supongamos dos soluciones S_1 y S_2 para la explicación:

- S_1 es considerada una solución con un HSI bajo, representa un hábitat con pocas especies. Su tasa de inmigración λ_1 va a ser más alta que λ_2 . Su tasa de emigración μ_1 va a ser más baja que μ_2 .
- S_2 es considerada una solución con un HSI alto, representa un hábitat con muchas especies. Su tasa de inmigración λ_2 va a ser más baja que λ_1 . Su tasa de emigración μ_2 va a ser más alta que μ_1 .

Se utilizan las tasas de emigración (μ) e inmigración (λ) de cada solución para compartir información probabilísticamente entre hábitats¹⁵. El proceso de modificación funciona así:

- La probabilidad de que una solución H_i sea seleccionada para ser modificada es proporcional a su tasa de inmigración λ_i .
- Si la solución H_i es seleccionada para ser modificada, se usa la tasa de emigración μ_j de las otras soluciones ($j \neq i$) para decidir probabilísticamente cuál de ellas donará un SIV.
- Se selecciona un SIV aleatorio de la solución donante H_j y este reemplaza a un SIV aleatorio en la solución receptora H_i .

La migración en BBO es un proceso adaptativo; se usa para modificar las "islas" (soluciones) existentes¹⁹. Además, como en otros algoritmos basados en población, se incorpora elitismo para retener las mejores soluciones, evitando que sean corrompidas por la inmigración.

2.1.2. Mutación

Los eventos aleatorios pueden cambiar drásticamente el HSI de un hábitat. Esto se modela en BBO como la mutación del SIV, y se usan las probabilidades de conteo de especies para determinar las tasas de mutación¹⁰. Las probabilidades de las especies se rigen por la Ecuación (2).

Cada miembro de la población tiene una probabilidad asignada (P_s), la cual hace referencia a la probabilidad que tenía a priori de existir como una solución al problema¹². Tanto las soluciones con un HSI muy alto como las soluciones con un HSI muy bajo son igualmente improbables¹³. Las soluciones con un HSI medio son, en comparación, muy probables.

Si una solución S tiene una baja probabilidad P_s , es sorprendente que exista y, por lo tanto, es probable que mute a otra solución. Por el contrario, una solución con una probabilidad alta es menos probable que mute. La siguiente fórmula define la tasa de mutación m , la cual es inversamente proporcional a la probabilidad de la solución:

$$m(S) = m_{\max} \left(\frac{1 - P_s}{P_{\max}} \right)$$

En esta fórmula, $m_{\max}$ es un parámetro definido por el usuario¹⁸. Este enfoque tiende a aumentar la diversidad entre la población¹⁹. Sin él, las soluciones más probables (HSI medio) tenderían a dominar la población. Este mecanismo hace que las soluciones con HSI bajo sean propensas a mutar (dándoles la oportunidad de mejorar) y que las soluciones con HSI alto también lo sean (dándoles la oportunidad de mejorar aún más).

Es importante recalcar que se utiliza un enfoque elitista. Se guardan las mejores soluciones antes de la mutación, de modo que si una mutación de alto riesgo arruina el HSI de una buena solución, la original se conserva. Se espera que las soluciones con HSI medio (las más probables) mejoren a través de la migración, por lo que se evita mutarlas (aunque conservan una pequeña probabilidad de mutación).

El mecanismo de mutación implementado depende del problema²⁴. Por ejemplo, en el problema de selección de sensores discutido en el artículo, la mutación consiste en reemplazar un sensor (SIV) escogido aleatoriamente por otro sensor generado también aleatoriamente.

2.2. Definiciones previas

El algoritmo está construido en base a una serie de definiciones preliminares que le dan forma a la estructura del mismo, pasando de un lenguaje basado en la biología con conceptos como hábitat a un lenguaje computacional. A continuación se presentan dichas definiciones:

Definición 1 *Un hábitat es un vector de $m \in \mathbb{N}$ enteros que representa una solución factible al problema.*

Definición 2 *El índice variable de idoneidad es un entero que es permitido en el hábitat.*

Definición 3 *El índice de idoneidad del hábitat $HSI : H \rightarrow \mathbb{R}$ es una medida de la bondad de la solución representada por el hábitat. En los algoritmos de optimización basados en población se denomina fitness.*

Definición 4 *Un ecosistema H^n es un grupo de $n \in \mathbb{N}$ hábitats.*

Definición 5 *La tasa de inmigración $\lambda : \mathbb{R} \rightarrow \mathbb{R}$ es una función monótona no creciente del HSI .*

Definición 6 *La tasa de emigración $\mu : \mathbb{R} \rightarrow \mathbb{R}$ es una función monótona no decreciente del HSI .*

En la práctica, asumimos que λ y μ son funciones lineales con los mismos máximos.

Definición 7 *La modificación del hábitat $\Omega : H^n \rightarrow H$ es un operador probabilístico que ajusta el hábitat H basado en el ecosistema H^n . La probabilidad de que H sea modificado es proporcional a la tasa de inmigración λ , y la probabilidad de que el origen de la modificación sea el hábitat H_j es proporcional a la tasa de emigración μ_j .*

Esta función se puede describir de la siguiente manera:

Definición 8 *Mutación $M : H \rightarrow H$ es un operador probabilístico que modifica de forma aleatoria un hábitat.*

Esta función se puede describir de la siguiente manera:

Definición 9 *La función de transición del ecosistema $\psi = (m, n, \lambda, \mu, \Omega, M) : H^n \rightarrow H^n$ es una séxtupla que modifica el ecosistema desde una iteración a la siguiente.*

Comienza calculando las tasas de inmigración y emigración para cada hábitat en el ecosistema. Luego la función de modificación del hábitat actúa sobre cada hábitat en el ecosistema, seguido por el cálculo del HSI. Y finalmente, se produce la mutación, seguido del cálculo del HSI para cada hábitat.

Definición 10 El algoritmo $BBO = (\mathcal{I}, \psi, \mathcal{T})$ es una tripleta que propone una solución a un problema de optimización, donde:

- $\mathcal{I} : \emptyset \rightarrow H^n, HSI^n$ es una función que crea un ecosistema inicial de hábitats y calcula cada HSI correspondiente.
- ψ es la función de transición del ecosistema.
- $\mathcal{T} : H^n \rightarrow \text{true}, \text{false}$ es un criterio de terminación.

2.3. Algoritmo

Después de toda una serie de definiciones que transforman conceptos biológicos a un lenguaje computacional, vamos a describir el algoritmo BBO de forma secuencial:

1. Inicializamos los parámetros del BBO.
2. Inicializamos un conjunto aleatorio de hábitats, donde cada hábitat se corresponde con una solución potencial al problema dado. Se trata de la implementación del operador $\mathcal{I}$.
3. Para cada hábitat, mapeamos el HSI al número de especies, S , la tasa de inmigración λ y la tasa de emigración μ .
4. Utilizamos la inmigración y emigración de forma probabilística para modificar cada hábitat no elitista, y recalculamos cada HSI.
5. Para cada hábitat, actualizamos la probabilidad de sus especies usando la siguiente ecuación:

$$\dot{P}_s = \begin{cases} -(\lambda_s + \mu_s)P_s + \mu_{s+1}P_{s+1}, & \text{si } s = 0, \\ -(\lambda_s + \mu_s)P_s + \lambda_{s-1}P_{s-1} + \mu_{s+1}P_{s+1}, & \text{si } 1 \leq s \leq S_{\text{máx}} - 1, \\ -(\lambda_s + \mu_s)P_s + \lambda_{s-1}P_{s-1}, & \text{si } s = S_{\text{máx}}. \end{cases}$$

A continuación mutamos cada hábitat no elitista basándonos en su probabilidad y recalculamos cada HSI.

6. Volvemos al paso (3) para la siguiente iteración. Este bucle termina después de cierto número de generaciones previamente definido o después de haber encontrado una solución aceptable para el problema. Esto es la implementación del operador $\mathcal{T}$.

2.4. Diferencias entre BBO y otros algoritmos de optimización basados en poblaciones

BBO es un algoritmo de optimización basado en poblaciones que no aplica reproducción o la generación de descendientes, lo que claramente lo diferencia de otros algoritmos como los algoritmos genéticos o estrategias evolutivas.

También se distingue de algoritmos como ACO (Optimización por Colonia de Hormigas) porque este último genera un nuevo conjunto de soluciones en cada iteración, mientras BBO mantiene el conjunto de soluciones de una iteración a la siguiente, basándose en la migración para adaptar probabilísticamente esas soluciones.

Las mayores características las comparte con algoritmos como PSO (Optimización por Enjambre de Partículas) y DE (Evolución Diferencial), ya que estos también mantienen una población fija de soluciones, pero cada solución es capaz de aprender de su entorno y adaptarse con el progreso del algoritmo. Sin embargo, BBO se diferencia de estos algoritmos en que sus soluciones cambian directamente según la migración desde otras soluciones. Luego, las soluciones de BBO comparten directamente sus atributos con otras soluciones.

2.5. Implementación

Para la implementación del algoritmo de BBO se ha decidido utilizar **Python 3** por su gran cantidad de librerías y su eficiencia en Jupyter Notebook. Se utilizaron librerías como **Numpy** para cálculos numéricos vectorizados, **Matplotlib.pyplot** para generar las gráficas de los frentes de Pareto, **random** para funciones de aleatoriedad, **copy** para crear clones de las soluciones candidatas y **scipy.special.comb** para el cálculo de coeficientes binomiales necesarios en el proceso de mutación.

El código se organizó en varios componentes en un Jupyter Notebook. En la primera parte del documento se han definido las seis funciones de prueba ($\mathcal{T}_1$ a $\mathcal{T}_6$) especificadas en el paper [2]. Cada función acepta un vector de solución y devuelve los dos objetivos f_1 y f_2 . El código permite los dos tipos de variables de las funciones de prueba: vectores de números decimales (**float**) para las funciones $\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, \mathcal{T}_4$ y $\mathcal{T}_6$, y la estructura especial de arrays de bits para la función $\mathcal{T}_5$.

Para representar una solución individual, se ha creado la clase **Habitat**. Esta clase guarda los valores de la solución (**sivs**), su valor de fitness (**hsi** - Habitat Suitability Index), y sus tasas de migración (λ y μ). Para crear esta clase hay que llamar a su constructor (**__init__**) el cual va a inicializar aleatoriamente los **sivs** según el **problem_type** ('float' o 'bits'), además los decimales están siendo vectorizados en su inicialización mediante **np.random.uniform** para decimales. Por último se ha creado un método **clone()** para duplicar instancias de forma segura usando **copy.deepcopy()**.

La lógica principal del algoritmo BBO se implementó en la función **run_bbo**. Dado que BBO es un algoritmo mono-objetivo, se adaptó para problemas multiobjetivo mediante una estrategia de suma ponderada: el **hsi** de cada hábitat se calculó como $w_1 \cdot f_1 + w_2 \cdot f_2$ para un par de pesos (w_1, w_2) dado. La función sigue el ciclo evolutivo estándar: evaluación del HSI, ordenación

de la población, aplicación de elitismo (conservando los `elitism_param` mejores), mapeo lineal de tasas de migración λ y μ basado en el ranking, y los operadores de migración y mutación. Para mejorar la eficiencia, las operaciones de migración para problemas que utilizan decimales se vectorizaron utilizando NumPy, aplicando las operaciones a todos los SIVs simultáneamente mediante máscaras booleanas y `np.where`. La mutación implementada sigue el esquema probabilístico dependiente de la probabilidad de existencia de la especie (P_s) descrito en el BBO original. Se precalculó el vector de probabilidades de especie P_s usando una distribución binomial con $S_{max} = \text{pop_size} - 1$. Luego, para cada hábitat no elitista, se calculó una tasa de mutación individual $m(S)$ usando la fórmula $m(S) = \text{m_max} \cdot (1 - P_s/P_{max})$ y se aplicó probabilísticamente a cada SIV (vectorizado para `float`, inversión de bit para `bits`). Al final de las generaciones, la función devuelve los valores (f_1, f_2) del mejor hábitat encontrado para los pesos utilizados.

Las simulaciones se realizaron adoptando un enfoque similar al de SOEA (Single-Objective Evolutionary Algorithm) con agregación. Para cada una de las seis funciones de prueba, se ejecutó `run_bbo` un total de 50 veces, cada vez con un par de pesos (w_1, w_2) distinto, obtenidos mediante `np.linspace` para variar w_1 de 0 a 1 y calculando $w_2 = 1 - w_1$. Cada una de estas 50 ejecuciones se realizó durante 100 generaciones. Los 50 puntos (f_1, f_2) resultantes (uno por cada par de pesos) se recopilaron y se graficaron como un diagrama de dispersión para visualizar la aproximación del frente de Pareto.

Los parámetros utilizados fueron consistentes en todas las simulaciones: tamaño de población (`POP_SIZE`) de 50, número de generaciones (`GENERATIONS`) de 100, parámetro de elitismo (`ELITISM_PARAM`) de 2, tasa máxima de migración (`MAX_MIGRATION`) de 1.0, y tasa máxima de mutación (`m_max`) de 0.05.

2.6. Conclusiones

En este apartado se redactan las conclusiones obtenidas de la implementación del algoritmo BBO adaptado utilizando la estrategia *Single-Objective Evolutionary Algorithm* (SOEA). Esta adaptación se ha realizado utilizando una suma ponderada para poder aplicarlo a los problemas multiobjetivos $\mathcal{T}_1, \dots, \mathcal{T}_6$ de [2].

En primer lugar, a pesar de que el algoritmo de Optimización Basada en Biogeografía está diseñado para problemas mono-objetivo, se puede adaptar para explorar el frente de Pareto en problemas multiobjetivo. Esta adaptación se ha realizado utilizando una suma ponderada de los pesos w_1 y w_2 , estos pesos van a ir cambiando a lo largo de las 50 ejecuciones. Estos pesos cumplen que

$$w_1 + w_2 = 1, \quad w_1 \geq 0, \quad w_2 \geq 0$$

Los resultados recogidos en la Figura 1 demuestran que para la mayoría de funciones, BBO consigue generar un conjunto de soluciones que se aproxima al frente de Pareto. El mayor aprendizaje que hemos obtenido realizando esta práctica se ha obtenido analizando el comportamiento de BBO en las diferentes funciones de prueba:

- En las funciones $\mathcal{T}_1$ Convexa y $\mathcal{T}_2$ No Convexa el algoritmo no ha tenido muchos problemas. En la función de prueba $\mathcal{T}_1$, este algoritmo generó puntos bastante cercanos al frente óptimo y con una buena distribución. Esto era esperable ya que las sumas ponderadas funcionan bien para frentes convexos. En cuanto a la función $\mathcal{T}_2$ el algoritmo consiguió dibujar la forma cóncava del frente aunque la distribución de puntos no es uniforme y hay menos densidad de soluciones en el centro.
- En la función ($\mathcal{T}_3$) sobre el Frente Discreto, se puede apreciar claramente la discontinuidad de esta función. Las soluciones generadas se agrupan en segmentos separados que componen el frente de Pareto. Podemos concluir que el algoritmo puede identificar y converger a regiones óptimas no continuas.
- Las funciones $\mathcal{T}_4$ Multimodal, $\mathcal{T}_5$ Deceptiva están diseñadas para representar Frentes Complejos. En la función $\mathcal{T}_4$ es donde se presentan las mayores dificultades ya que existen pocas soluciones cercanas al frente de Pareto. En cuanto a $\mathcal{T}_5$ ciertas combinaciones de bits alejan la solución del óptimo global. Con la población de 50 y 100 generaciones, podemos concluir basándonos en los resultados de las gráficas que BBO ha convergido a un frente subóptimo. Es probable que se necesiten poblaciones más grandes, más generaciones, o quizás ajustes en los parámetros de migración/mutación para superar este problema.
- En la función $\mathcal{T}_6$ para representar un Frente no Uniforme, el algoritmo ha conseguido representar la forma general del frente. Aunque la densidad de los puntos no es uniforme y existen muchas soluciones en los extremos de f_1 , este es el comportamiento esperado de esta función. Esto se debe a que es menos probable que las operaciones de migración y mutación del algoritmo generen soluciones en las zonas intermedias por el rango estrecho de valores x_1 debido a la forma de la función $\sin^6(6\pi x_1)$.

Mediante la realización de esta práctica, hemos comprendido en profundidad el algoritmo y la analogía que tiene inspirada en la naturaleza y la biología. La implementación de las funciones de prueba ha sido importante para poder ver el rendimiento de este algoritmo en diferentes problemas y poder comprender en qué casos funciona mejor. Ver como BBO se enfrenta a estos problemas *benchmark* nos ha ayudado a comprender y relacionar la teoría vista en clase con la práctica computacional.

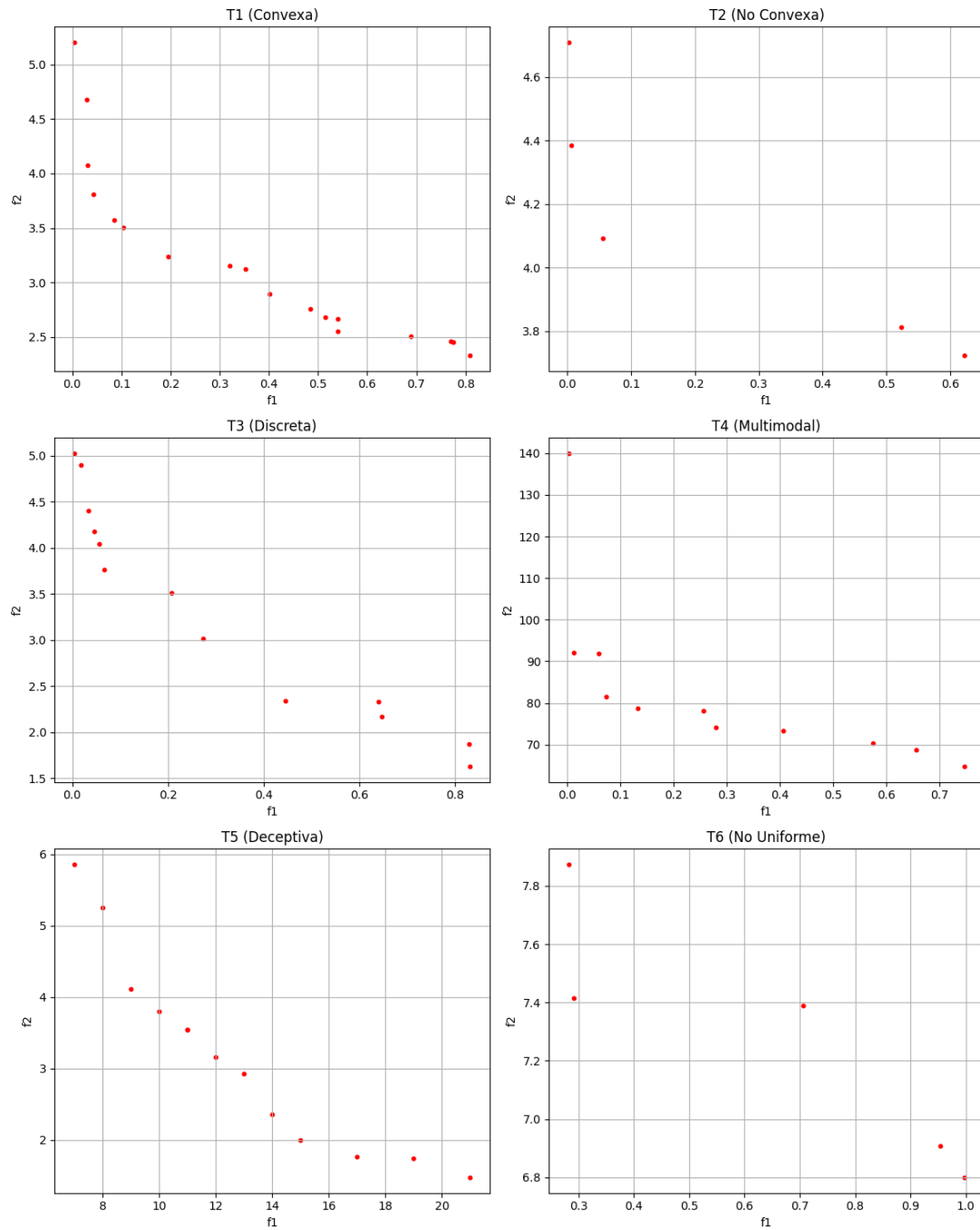


Figura 1: Frentes de Pareto aproximados obtenidos por BBO-SOEA para las seis funciones de prueba T1 a T6.

3. Sistema de hormigas Elitista

3.1. Introducción

Uno de los problemas más estudiados por los etólogos ha sido entender cómo animales con capacidades visuales limitadas, como las hormigas, logran establecer rutas eficientes desde su colonia para conseguir recursos y volver. Se ha descubierto que la forma más usual para comunicar información entre los individuos sobre los caminos, y que se usa para decidir a dónde ir, se basa en el rastro de feromonas. El movimiento de hormigas deposita esta sustancia en el suelo, marcando el sendero. Este mecanismo genera un proceso de retroalimentación positiva: de tal forma que cuanto más sigan las hormigas un rastro, más atractivo se vuelve ese rastro para ser seguido. En concreto, este comportamiento colectivo se caracteriza porque la probabilidad de que una hormiga escoja un camino aumenta en función del número de individuos que ya han transitado esa misma ruta.

3.2. Sistema de Hormigas

En primer lugar vamos a utilizar el problema del viajante de comercio (TSP) como caso de estudio para implementar nuestro sistema de hormigas elitista. El TSP es un problema clásico de optimización combinatoria que busca encontrar la ruta más corta que visite un conjunto de ciudades exactamente una vez y regrese a la ciudad de origen.

Dado un conjunto de n ciudades, denominamos d_{ij} a la longitud del camino entre las ciudades i y j , que suele ser la distancia euclídea. Este conjunto de ciudades y caminos viene representado por un grafo (N, E) , donde N es el conjunto de nodos (ciudades) y E el conjunto de aristas (caminos entre ciudades).

Denominamos $b_i(t)$ el número total de hormigas en la ciudad i en el tiempo t y $m = \sum_{i=1}^n b_i(t)$ el número total de hormigas en el tiempo t . Cada hormiga dispone de las siguientes características:

- Escoge la ciudad siguiente con una probabilidad, que es función de la cantidad de feromona en el camino y de la distancia a la ciudad siguiente.
- Obligamos a que realice recorridos válidos, prohibiendo las transiciones a pueblos ya visitados hasta que se complete un recorrido, mediante una lista tabú.
- Cuando se completa un recorrido, deposita una sustancia llamada rastro en cada arista visitada.

Sea $\tau_{ij}(t)$ la cantidad de feromona en el camino entre las ciudades i y j en el tiempo t . Cada hormiga en el tiempo t , debe escoger la siguiente ciudad, donde estará en tiempo $t+1$. Por lo tanto, si llamamos a una iteración del algoritmo a los m movimientos realizados por las m hormigas en el intervalo $(t, t+1)$, entonces cada n iteraciones del algoritmo, cada hormiga ha completado un recorrido, lo que denominaremos ciclo.

En cada ciclo la intensidad del camino se actualiza de la siguiente forma:

$$\tau_{ij}(t+n) = \rho \cdot \tau_{ij}(t) + \Delta\tau_{ij} \quad (1)$$

donde $0 < \rho < 1$ es un coeficiente tal que $1 - \rho$ representa la evaporación del camino entre t y $t+n$, y

$$\Delta\tau_{ij} = \sum_{k=1}^m \Delta\tau_{ij}^k \quad (2)$$

donde $\Delta\tau_{ij}^k$ es la cantidad por unidad de longitud de feromona que la hormiga k deposita en el camino entre las ciudades i y j , entre el tiempo t y $t+n$. Esta cantidad se define como:

$$\Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{si la hormiga } k \text{ utiliza el camino } (i, j) \text{ en su recorrido} \\ 0 & \text{en otro caso} \end{cases} \quad (3)$$

donde Q es una constante y L_k la longitud del recorrido realizado por la hormiga k .

Con el objetivo de satisfacer la restricción de que cada hormiga debe visitar todas las n diferentes ciudades, asociamos a cada hormiga una lista tabú que almacena las ciudades ya visitadas por esa hormiga. Cuando se completa un ciclo, la lista tabú se utiliza para calcular la solución actual de cada hormiga. Luego, la lista se vacía y cada hormiga comienza un nuevo ciclo.

Llamamos visibilidad, n_{ij} , a la distancia inversa entre las ciudades i y j , $\frac{1}{d_{i,j}}$. Esta cantidad no se modifica durante la ejecución del algoritmo y representa la información heurística disponible para las hormigas.

Definimos la probabilidad de transición de una ciudad i a una ciudad j para la hormiga k en el tiempo t como:

$$p_{ij}^k = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha \cdot [n_{ij}]^\beta}{\sum_{l \in \text{allowed}_k} [\tau_{il}(t)]^\alpha \cdot [n_{il}]^\beta} & \text{si } j \in \text{allowed}_k \\ 0 & \text{en otro caso} \end{cases} \quad (4)$$

donde allowed_k es el conjunto de ciudades que la hormiga k no ha visitado aún, y α y β son parámetros que controlan la importancia relativa entre la feromona y la visibilidad. Por lo tanto, esta probabilidad es un equilibrio entre la visibilidad, que indica que se deben elegir las ciudades más cercanas con mayor probabilidad, y la intensidad del rastro en el tiempo t , que indica que si en la arista (i, j) ha habido mucho tráfico, entonces ese camino es más atractivo.

3.3. Algoritmo

El algoritmo comienza con una fase de inicialización, donde las hormigas se posicionan en diferentes ciudades y se establece la intensidad inicial de feromona en cada arista del grafo mediante $\tau_{ij}(0)$. Inicializamos el primer elemento de la lista tabú de cada hormiga con la ciudad donde se encuentra.

Luego, cada hormiga se moverá desde la ciudad i a la ciudad j según la probabilidad de transición y tras n iteraciones todas las hormigas habrán completado un ciclo y sus listas tabú contendrán un recorrido completo. En este punto, para cada hormiga k se actualiza L_k y la

intensidad de feromona en cada arista del grafo según la fórmula dada anteriormente. Además, se guarda el camino más corto encontrado, $\min_k L_k$, $k = 1, \dots, m$. Este proceso se repite hasta que se cumple un criterio de parada, como alcanzar un número máximo de ciclos, NC_{MAX} o todas las hormigas convergen a la misma solución.

Formalmente, el algoritmo se puede describir con los siguientes pasos:

1. Inicialización:

Definimos $t := 0$.

Definimos $NC := 0$.

Para cada arco (i, j) definimos la intensidad $\tau_{ij}(0) = c$ y $\Delta\tau_{ij} = 0$.

Colocar las m hormigas en los n nodos.

2. Sea $s := 1$.

Para $k = 1$ hasta m hacer

Colocar la ciudad inicial de la hormiga k en la lista tabú, $tabu_k(s)$.

3. Repetir hasta que la lista tabú se complete:

Definimos $s := s+1$

Para $k = 1$ hasta m hacer

Elegir la ciudad j a la que moverse con probabilidad $p_{ij}^k(t)$.

Mover la hormiga k a la ciudad j .

Colocar la ciudad j en la lista tabú, $tabu_k(s)$.

4. Para $k = 1$ hasta m hacer

Mover la hormiga k de $tabu_k(n)$ a $tabu_k(1)$.

Calcular la longitud del recorrido descrito por la hormiga k , L_k .

Actualizar el camino más corto encontrado.

Para cada arco (i, j) del grafo hacer

Para $k = 1$ hasta m hacer

$$\Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{si } (i, j) \in \text{ciclo descrito por } tabu_k \\ 0 & \text{en otro caso} \end{cases}$$

$$\Delta\tau_{ij} := \Delta\tau_{ij} + \Delta\tau_{ij}^k$$

5. Para cada arco (i, j) calcular $\tau_{ij}(t+n) = \rho \cdot \tau_{ij}(t) + \Delta\tau_{ij}$.

Definir $t := t + n$.

Definir $NC := NC + 1$.

Para cada arco (i, j) definir $\Delta\tau_{ij} := 0$.

6. Si $(NC < NC_{MAX})$ y no todas las hormigas han convergido a la misma solución entonces

Vaciar todas las listas tabú.

Ir al paso 2.

sino

Imprimir el ciclo más corto encontrado.

Parar.

3.3.1. Estrategia elitista

Para mejorar el rendimiento del algoritmo, implementamos una estrategia elitista. Para ello añadimos al camino de cada arco del mejor ciclo la cantidad $e \cdot Q/L^*$, donde e es el número de hormigas elitistas y L^* la longitud del mejor ciclo encontrado hasta el momento. De esta forma, se refuerza el camino del mejor ciclo, acelerando la convergencia del algoritmo.

3.4. Implementación

Para validar el funcionamiento del Sistema de Hormigas Elitista, se ha realizado una implementación práctica en lenguaje Python. El desarrollo parte de los requisitos establecidos en el enunciado de la Práctica 9, evolucionando desde la resolución del problema base hacia una propuesta experimental multi-objetivo.

3.4.1. Descripción del Problema y Entorno

El problema a resolver es el **Problema de Asignación Generalizado** descrito en el enunciado de la práctica. Este escenario plantea la necesidad de asignar un conjunto de $N = 4$ tareas, denotadas como T_i ($i = \{1, \dots, 4\}$), a un conjunto de $M = 5$ operarios, denotados como O_j ($j = \{1, \dots, 5\}$).

El modelo está sujeto a las siguientes restricciones y parámetros definidos en la documentación:

- **Restricción de Unicidad:** Cada tarea debe ser realizada por un único operario.
- **Restricción de Capacidad:** Aunque un operario puede realizar múltiples tareas, la suma de los tiempos requeridos por estas (r_i) no puede exceder el tiempo total disponible de dicho operario (d_j).
- **Función de Coste:** Existe un coste c_{ij} asociado a que el operario j realice la tarea i .

El objetivo fundamental estipulado es encontrar la asignación válida que minimice el coste total. No obstante, en esta implementación hemos decidido ir un paso más allá de lo solicitado: además de resolver el problema mono-objetivo original, hemos introducido artificialmente un segundo objetivo de optimización (equilibrio de carga) para testar la flexibilidad del algoritmo en entornos multi-objetivo.

Los datos utilizados se corresponden fielmente con la tabla proporcionada en el enunciado, modelados computacionalmente mediante NumPy:

$$C_{ij}(\text{Costes}) = \begin{pmatrix} 2 & 3 & 4 & 1 \\ 3 & 2 & 3 & 2 \\ 2 & 2 & 1 & 2 \\ 3 & 3 & 3 & 3 \\ 2 & 1 & 2 & 2 \end{pmatrix} \quad (5)$$

$$r_i(\text{T. Requerido}) = [3, 2, 2, 3], \quad d_j(\text{T. Disponible}) = [4, 5, 3, 4, 4] \quad (6)$$

3.4.2. Enfoque mono-objetivo: (Elitist Ant System)

En esta primera fase, nos ceñimos estrictamente al objetivo del enunciado: minimizar el coste total ($f_1 = \sum c_{ij}$). Se implementó el algoritmo descrito en el apartado 3.3, definiendo la información heurística η_{ij} como la inversa del coste ($1/c_{ij}$), tal que las hormigas tiendan a escoger las asignaciones más económicas. Se aplicó la estrategia elitista para reforzar el mejor camino global encontrado en cada iteración.

3.4.3. Extensión multi-objetivo (MOACO y MAS)

Para extender la práctica y analizar el comportamiento de las metaheurísticas ante conflictos de objetivos, definimos un segundo objetivo no presente en el enunciado original:

- **Objetivo 1** (f_1): Minimizar Coste Total (Objetivo original).
- **Objetivo 2** (f_2): Minimizar el Desequilibrio de Carga (Desviación típica del tiempo de trabajo entre operarios).

Se implementaron dos variantes para resolver este problema bi-objetivo:

1. **Simple MOACO**: Utiliza un archivo externo para almacenar el Frente de Pareto.
2. **MAS (Multi-Objective Ant System)**: Asigna pesos variables λ a cada hormiga para equilibrar la importancia de ambos objetivos.

3.4.4. Resultados Experimentales

El primer análisis se realizó utilizando estrictamente las matrices de costes y tiempos proporcionadas en el enunciado de la Práctica 9. Tras ejecutar tanto la variante Mono-objetivo como las variantes Multi-objetivo (MOACO y MAS), se observó un fenómeno de convergencia unificada.

- **Solución Obtenida**: Todos los algoritmos convergieron a la misma solución global:

- Coste Total (f_1): **6** (Mínimo teórico).
- Desequilibrio (f_2): **1.0954** (Desviación típica).

Este comportamiento se debe a la topología específica de los datos de entrada. En esta instancia del problema GAP, los objetivos resultaron ser no conflictivos. La asignación de tareas a los operarios más eficientes en costes minimizando f_1 generó, de manera colateral, una distribución de carga que no violaba las restricciones y mantenía un desequilibrio bajo. Al no existir conflicto entre ahorrar dinero y equilibrar carga, el Frente de Pareto teóricos colapsó en un único punto dominante, impidiendo visualizar una curva de compromiso trade-off real.

Uno de los aspectos críticos en la validación del algoritmo es asegurar que la solución encontrada no solo minimiza la función objetivo, sino que respeta estrictamente las restricciones de capacidad del problema GAP. A primera vista, una inspección de la matriz de costes C_{ij} sugeriría que el coste mínimo teórico podría ser $f_1 = 5$. Este valor se obtiene aplicando una estrategia *greedy*, seleccionando el mínimo absoluto de cada tarea sin considerar el consumo de recursos. Sin embargo, como se ilustra en la Figura 2, esta combinación es inviable:

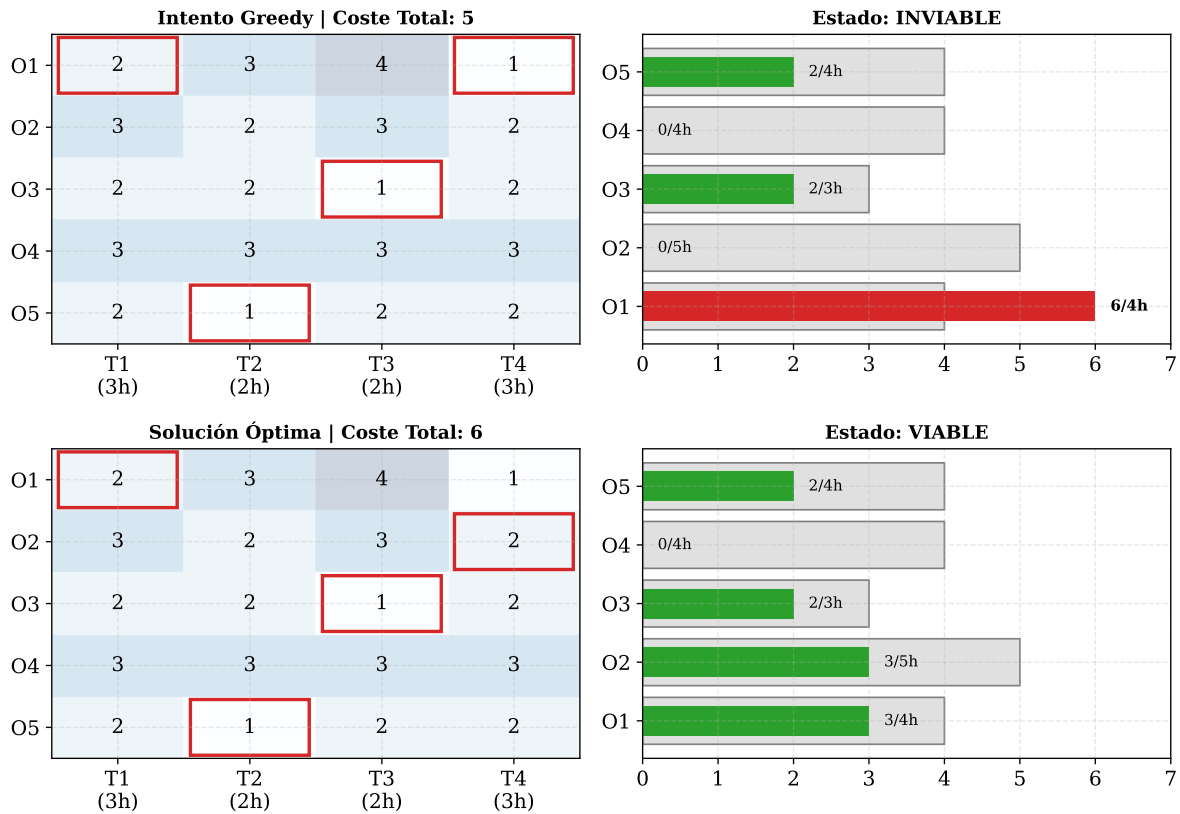


Figura 2: Comparativa entre la selección Greedy y la solución óptima. Sumando los mínimos absolutos viola la capacidad del O_1 .

Por lo tanto, la solución mínima viable de este problema es $f_1 = 6$ tan y como han indicado los tres algoritmos implementados en la práctica. A continuación decidimos crear otro escenario donde las habilidades de los operarios eran menos regulares dando a lugar a un frente de pareto para los algoritmos multiobjetivo.

Validación Adicional: Escenario de Conflicto

Dado que los datos originales no permitían apreciar la capacidad del algoritmo para gestionar objetivos contrapuestos, se diseñó un segundo escenario experimental. El objetivo fue forzar la aparición de un Frente de Pareto real modificando los datos de entrada para generar un conflicto explícito entre Coste y Equilibrio. Se definió un problema con $N = 8$ tareas y $M = 3$ operarios con perfiles extremos:

- **Operario 1 (barato):** Coste muy bajo ($c = 1$) pero alta probabilidad de saturación.
- **Operario 2 (medio):** Coste moderado ($c = 4$).
- **Operario 3 (caro):** Coste alto ($c = 8$), su uso es necesario para equilibrar la carga pero penaliza el coste.

Al ejecutar el algoritmo MOACO sobre este nuevo conjunto de datos, se obtuvo un conjunto de soluciones no dominadas diverso, tal como se muestra en la Figura 3.

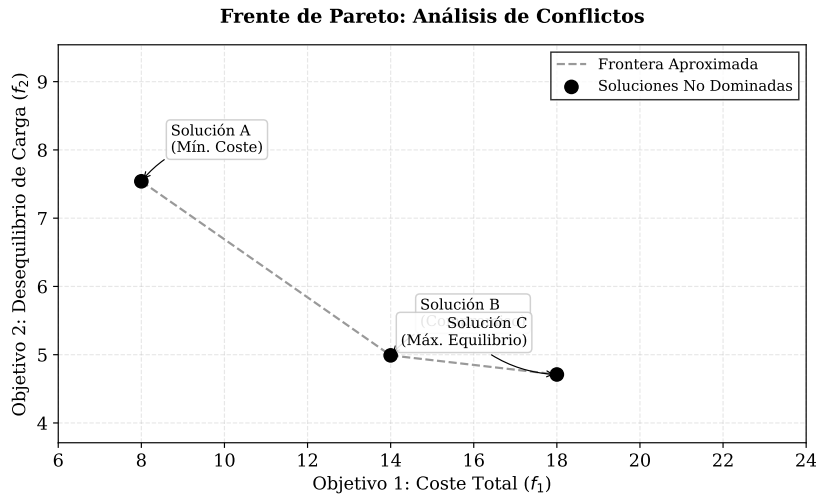


Figura 3: Frente de Pareto obtenido en el escenario de conflicto. Se observa la curva de compromiso entre el Coste Total y el Desequilibrio de Carga.

A diferencia del caso anterior, aquí se observan claramente tres regiones de decisión:

1. **Región de mínimo coste** ($f_1 \approx 8$): El algoritmo asigna casi todo al Operario 1. El coste es mínimo, pero el desequilibrio (f_2) se dispara por encima de 7.5.
2. **Región de compromiso** ($f_1 \approx 14$): Se empieza a derivar trabajo al Operario 2. El coste sube, pero el desequilibrio mejora notablemente.
3. **Región de máximo equilibrio** ($f_1 \approx 18$): Se utiliza al Operario 3. Es la opción más cara, pero logra la distribución de trabajo más equitativa posible ($f_2 < 4,8$).

La segunda fase experimental valida la robustez de la implementación, demostrando su capacidad para mantener la diversidad poblacional y hallar soluciones de compromiso en escenarios de conflicto real, superando así las limitaciones observadas en los datos originales. Por otro lado, el análisis de la Figura 2 justifica estructuralmente el resultado obtenido: mientras que una estrategia greedy alcanzaría un coste teórico de 5 a expensas de violar la capacidad del Operario 1 (quien recibiría 6 horas de carga frente a sus 4 disponibles), el sistema converge correctamente hacia el óptimo factible de coste 6, priorizando el cumplimiento estricto de las restricciones mediante la reasignación eficiente de recursos.

4. FANS: Fuzzy Adaptative Neighborhood Search

4.1. Introducción

Como explican los autores [3], FANS es un método de búsqueda por entornos adaptativo y difuso, basado en cuatro componentes principales:

- Operador: construye las soluciones.
- Valoración difusa: cualifica las soluciones.
- Administrador de operación: adapta el comportamiento o características del operador.
- Administrador de vecindario:

De forma más matemática FANS queda definido por la septúpla $(NS, O, OS, \mu, Pars, (cond, accion))$, donde:

- NS: administrador de vecindario.
- O: operador utilizado para construir soluciones.
- OS: administrador de operación.
- μ : valoración difusa.
- Pars: conjunto de parámetros.
- $(cond, accion)$: regla de tipo IF cond THEN accion que se utiliza para detectar y actuar en consecuencia cuando la búsqueda se ha estancado.

4.1.1. Operador de Modificación

El operador de modificación va a construir nuevas soluciones s_i a partir de las soluciones anteriores s . Se define tal que $\mathcal{O}_i \in \mathcal{M}$, $\mathcal{O}_i : \mathcal{S} \times \mathcal{P} \rightarrow \mathcal{S}$

4.1.2. Valoración Difusa

Las soluciones se evalúan mediante la función objetivo y una valoración difusa. Esta valoración difusa se define tal que: $\mu \in \mathcal{F}$, con $\mu : \mathcal{R} \rightarrow [0, 1]$. Esta valoración μ nos devuelve el grado de pertenencia de la solución al conjunto difuso.

4.1.3. Utilidad

La utilidad fundamental de la valoración difusa reside en su capacidad para modular el comportamiento global del algoritmo FANS. La definición de sus componentes y la interacción entre ellos permiten que el algoritmo emule diversas metaheurísticas clásicas simplemente ajustando la definición de *aceptabilidad*.

Asumiendo un esquema de selección de tipo *First* (donde se selecciona la primera solución vecina $\hat{s}$ tal que $\mu(s, \hat{s}) \geq \lambda$, siendo λ un nivel mínimo de calidad), la valoración difusa permite adaptar la estrategia de búsqueda a los siguientes comportamientos clásicos en un problema de minimización:

- **Caminos Aleatorios:** Si se considera cualquier solución del vecindario como aceptable, el algoritmo explora el espacio de búsqueda sin dirección preferente. Esto se logra definiendo una valoración constante máxima:

$$\mu(s, \hat{s}) = 1, \quad \forall \hat{s} \in N(s) \quad (7)$$

De esta forma, cualquier solución generada es seleccionada, produciendo un comportamiento puramente estocástico.

- **Hill Climbing:** Para obtener un comportamiento de búsqueda local estricta, la valoración difusa debe restringir la aceptabilidad únicamente a aquellas soluciones que mejoran el costo actual. Fijando $\lambda = 1$, la función se define como:

$$\mu(s, \hat{s}) = \begin{cases} 1 & \text{si } f(\hat{s}) < f(s) \\ 0 & \text{en otro caso} \end{cases} \quad (8)$$

Bajo esta configuración, el algoritmo solo transita hacia soluciones mejores, terminando su ejecución al alcanzar un óptimo local.

- **Recocido Simulado:** Este método permite aceptar soluciones que empeoran la función objetivo con cierta probabilidad para escapar de óptimos locales. En FANS, esto se modela mediante una función de valoración que depende de un umbral de deterioro β :

$$\mu(s, \hat{s}) = \begin{cases} 0 & \text{si } f(\hat{s}) > \beta \\ \frac{\beta - f(\hat{s})}{\beta - f(s)} & \text{si } f(s) \leq f(\hat{s}) \leq \beta \\ 1 & \text{si } f(\hat{s}) < f(s) \end{cases} \quad (9)$$

Aquí, β actúa como un límite de aceptabilidad. Si β se define como una función dinámica $h(s, \hat{s}, t)$ que depende del tiempo o del número de iteraciones t , es posible reducir el deterioro aceptable a medida que la búsqueda progresa (enfriamiento), convergiendo finalmente hacia un comportamiento donde solo se aceptan mejoras.

En conclusión, mediante la variación de μ y los parámetros asociados, la valoración difusa transforma a FANS en un algoritmo de umbral adaptable capaz de replicar la dinámica cualitativa de métodos de búsqueda local consolidados.

4.1.4. Administrador de Operación

El administrador de Operación $\mathcal{OS}$ es el responsable de definir el orden de aplicación de los diferentes operadores utilizados. $\mathcal{OS}(\mathcal{O}^{ti}) \Rightarrow \mathcal{O}^{tj}$. Otra opción es disponer de un conjunto de operadores de modificación $\{\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_k\}$ donde el Administrador puede escoer el orden de operación de los mismos.

4.1.5. Administrador de Vecindario

El Administrador del vecindario $\mathcal{N}$ es el responsable de generar nuevas soluciones. Su definición matemática es $\mathcal{NS} : \mathcal{S} \times \mathcal{F} \times \mathcal{M} \times \mathcal{P} \rightarrow \mathcal{S}$. Este vecindario puede ser:

- Operativo: $\mathcal{N}(s) = \{\hat{s}_i \mid \hat{s}_i = \mathcal{O}_i(s)\}$
- Semántico: $\hat{\mathcal{N}}(s) = \{\hat{s}_i \in \mathcal{N}(s) \mid \mu(\hat{s}_i) \geq \lambda\}$. Las soluciones adecuadas son las que superen un cierto λ

4.1.6. Parámetros globales

El algoritmo mantiene un conjunto $Pars \in \mathcal{P}$ para llevar un registro de los estados de la búsqueda. Este conjunto es utilizado

4.2. Algoritmo

En cada iteración se comienza con una llamada al administrador de vecindario $\mathcal{NS}$ con los parámetros $\mathcal{S}_{cur}$, que representa la solución actual, μ , valoración difusa, y $\mathcal{O}$, operador de modificación.

El resultado de la ejecución del administrador da lugar a dos posibles situaciones: se ha encontrado la solución óptima, en términos de μ o no. En el primer caso esa solución obtenida pasa a ser la nueva solución actual y los parámetros de μ son adaptados. En caso contrario, se aplica un mecanismo de escape, donde se ejecuta el administrador de operación $\mathcal{OS}$, con el objetivo de devolver una versión modificada del operador $\mathcal{O}$, lo que influirá en la búsqueda de soluciones.

Por último, existe una condición para evitar el estancamiento. Si se han utilizado varios operadores distintos y no se ha obtenido ninguna mejora, se evaluará el costo de la nueva solución y se adaptará la valoración difusa, reiniciando posteriormente la búsqueda.

El algoritmo finaliza cuando el número de evaluaciones de la función de costo alcanza cierto límite o cuando se realizó un número predeterminado de evaluaciones.

El pseudocódigo es el siguiente:

Algorithm 1 Procedimiento FANS

```

Begin
  /*  $\mathcal{O}$  : el operador de modificacion */
  /*  $\mu()$  : la valoracion difusa */
  /*  $S_{cur}, S_{new}$  : soluciones */
  /* NS: el administrador de vecindario */
  /* OS: el administrador de operacion */
  Inicializar Variables();
  while ( not-fin ) do
    /* Ejecutar Admin. de Vecindario NS */
     $S_{new} = \text{NS}(S_{cur}, \mu(), \mathcal{O})$ ;
    if ( $S_{new}$  es “buena” en base a  $\mu()$ ) then
       $S_{cur} := S_{new}$ ;
      adaptar-Valoracion-Difusa( $\mu(), S_{cur}$ );
    else
      /* Fue imposible hallar una  $S_{new}$  “buena”
         con el operador  $\mathcal{O}$  : Sera modificado */
       $\mathcal{O} := \text{OS-ModificarOperador}(\mathcal{O})$ ;
    end if
    if (HayEstancamiento?()) then
      Escape();
    end if
  end while
End.

```

4.3. FANS como Heurística de Propósito General

Más allá de su capacidad para mimetizar métodos clásicos, FANS se constituye como un *framework* de optimización en sí mismo. Su mayor fortaleza radica en que la valoración difusa (μ) no es simplemente una función matemática, sino que modela el **criterio de un decisor** ante el problema.

Esta flexibilidad permite definir curvas de aceptabilidad personalizadas que reflejan estrategias de búsqueda complejas, imposibles de configurar en algoritmos rígidos. A continuación se describen tres perfiles de decisión que ilustran esta capacidad:

[h]

- **Criterio de Diversificación:** Representa a un decisor que busca soluciones de costo similar a la actual ($f(\hat{s}) \approx f(s)$) pero diferentes en estructura. Es ideal cuando existen criterios cualitativos difíciles de cuantificar (como aspectos estéticos), permitiendo generar un abanico de alternativas equivalentes en costo antes de tomar una decisión final.
- **Criterio de Insatisfacción:** Refleja la postura de un decisor que no tolera la solución actual. La valoración difusa asigna el máximo grado de aceptabilidad a cualquier opción que ofrezca una mejora, incentivando cambios rápidos en la trayectoria de búsqueda.
- **Criterio Conservador:** Corresponde a un decisor reacio al cambio. Para que una nueva

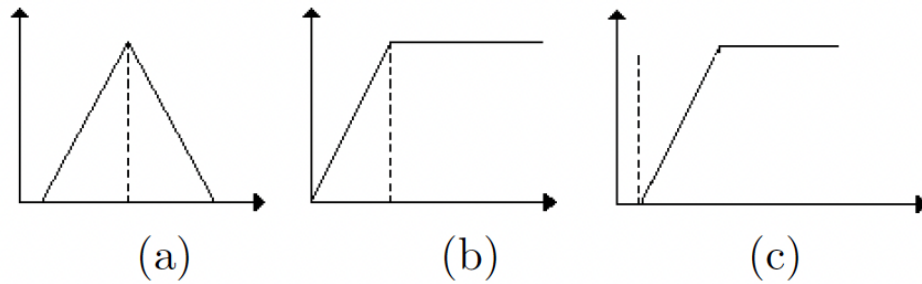


Figura 4: Ejemplo de variaciones difusas.

solución desplace a la actual, no basta con una mejora marginal; se exige que la mejora sea considerable y sustancial.

Adicionalmente, la potencia de FANS se ve amplificada por la posibilidad de aplicar **modificadores lingüísticos** (como *muy* o *algo*) sobre las definiciones de valoración, permitiendo un ajuste fino del comportamiento del algoritmo.

En síntesis, FANS permite variar la estrategia de exploración de forma intuitiva y comprensible. Dado que diferentes topologías de problemas requieren diferentes estrategias de búsqueda para obtener resultados óptimos, esta capacidad de adaptación convierte a FANS en una herramienta genérica de optimización sumamente robusta.

5. Conclusiones

Bibliografía

- [1] D. Simon, “Biogeography-based optimization,” *IEEE transactions on evolutionary computation*, vol. 12, no. 6, pp. 702–713, 2008.
- [2] E. Zitzler, K. Deb, and L. Thiele, “Comparison of multiobjective evolutionary algorithms: Empirical results,” *Evolutionary computation*, vol. 8, no. 2, pp. 173–195, 2000.
- [3] A. Blanco, D. A. Pelta, and J. L. Verdegay, “Fans: una heurística basada en conjuntos difusos para problemas de optimización,” *Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial*, vol. 7, no. 19, p. 0, 2003.